

# Shiv Nadar University Chennai

|                     |                                               |             |
|---------------------|-----------------------------------------------|-------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY: 2026–27 |

## Experiment 1

### Implementation of a Single Layer Perceptron for Binary Classification

#### Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

#### Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

#### Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$  : Input vector
- $\mathbf{w}$  : Weight vector
- $b$  : Bias
- $z$  : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where  $\eta$  denotes the learning rate.

## Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

### Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

| Activation | Output Range        | Typical Applications                    |
|------------|---------------------|-----------------------------------------|
| Step       | $\{0, 1\}$          | Classical Perceptron                    |
| Sigmoid    | $(0, 1)$            | Binary Classification Output Layer      |
| Tanh       | $(-1, 1)$           | Hidden Layers (Traditional Networks)    |
| ReLU       | $[0, \infty)$       | Hidden Layers in CNNs and MLPs          |
| Leaky ReLU | $(-\infty, \infty)$ | Avoiding Dying ReLU                     |
| Softmax    | $(0, 1)$            | Multi-class Classification Output Layer |

**Note:** This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

## Dataset Description

**Dataset:** Banknote Authentication Dataset

**Source:** UCI Machine Learning Repository

**Download Link:**

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

|                |                       |
|----------------|-----------------------|
| Instances      | 1372                  |
| Features       | 4 Numerical Features  |
| Classes        | 2                     |
| Missing Values | None                  |
| Task           | Binary Classification |

### Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

## Experimental Procedure

### Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

#### Expected Output

Dataset summary and statistical information.

### Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

### Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

#### **Task 4: Perceptron Implementation**

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

#### **Task 5: Model Training**

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

#### **Task 6: Model Evaluation**

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

## **Source Code**

GitHub Repository: [https://github.com/un-shreeya/DeepLearning\\_Lab](https://github.com/un-shreeya/DeepLearning_Lab)

## Experimental Results

### Dataset Information

|                  |                      |
|------------------|----------------------|
| Total Samples    | 1372                 |
| Input Features   | 4                    |
| Target Classes   | 2 (Fake and Genuine) |
| Training Samples | 1097                 |
| Testing Samples  | 275                  |

### Model Performance

|           |        |
|-----------|--------|
| Metric    | Value  |
| Accuracy  | 98.55% |
| Precision | 98.43% |
| Recall    | 98.43% |
| F1-Score  | 98.43% |

### Final Learned Parameters

|               |                                             |
|---------------|---------------------------------------------|
| Final Weights | [-0.247649, -0.228401, -0.253429, 0.007397] |
| Final Bias    | 0.31                                        |

### Learning Rate Comparison

| Learning Rate | Accuracy | Epochs Used |
|---------------|----------|-------------|
| 0.001         | 98.55%   | 100         |
| 0.01          | 98.91%   | 100         |
| 0.1           | 98.55%   | 100         |

# Generated Plots and Interpretation

## 1. Feature Histograms

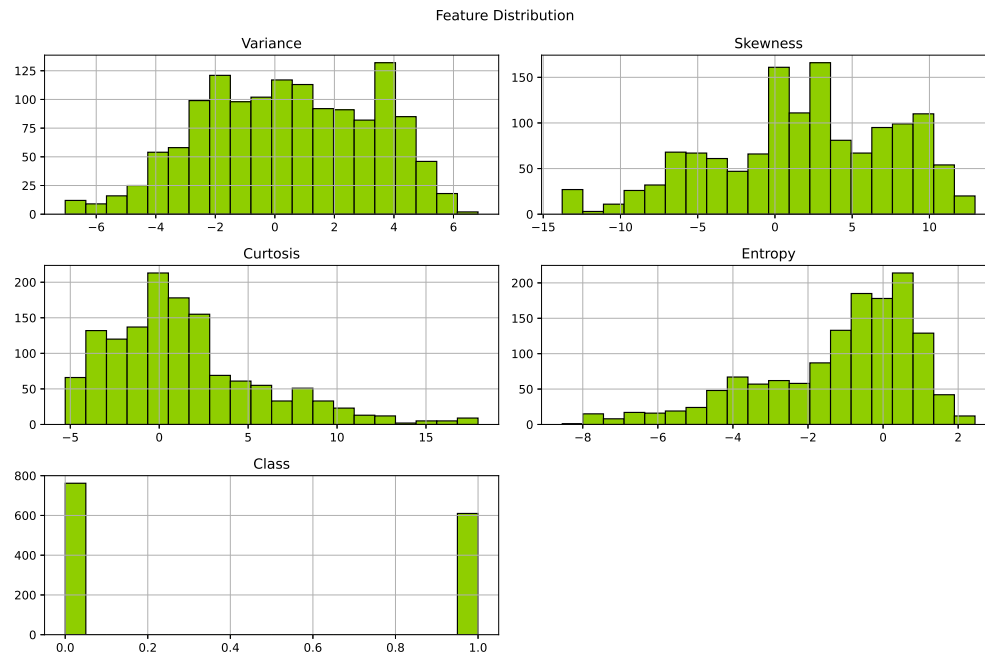


Figure 1: Feature Histograms

**Inference:** Variance and Skewness look somewhat distributed equally. For Class, distribution is binary. Kurtosis plot is right skewed and Entropy is left skewed.

## 2. Correlation Heatmap

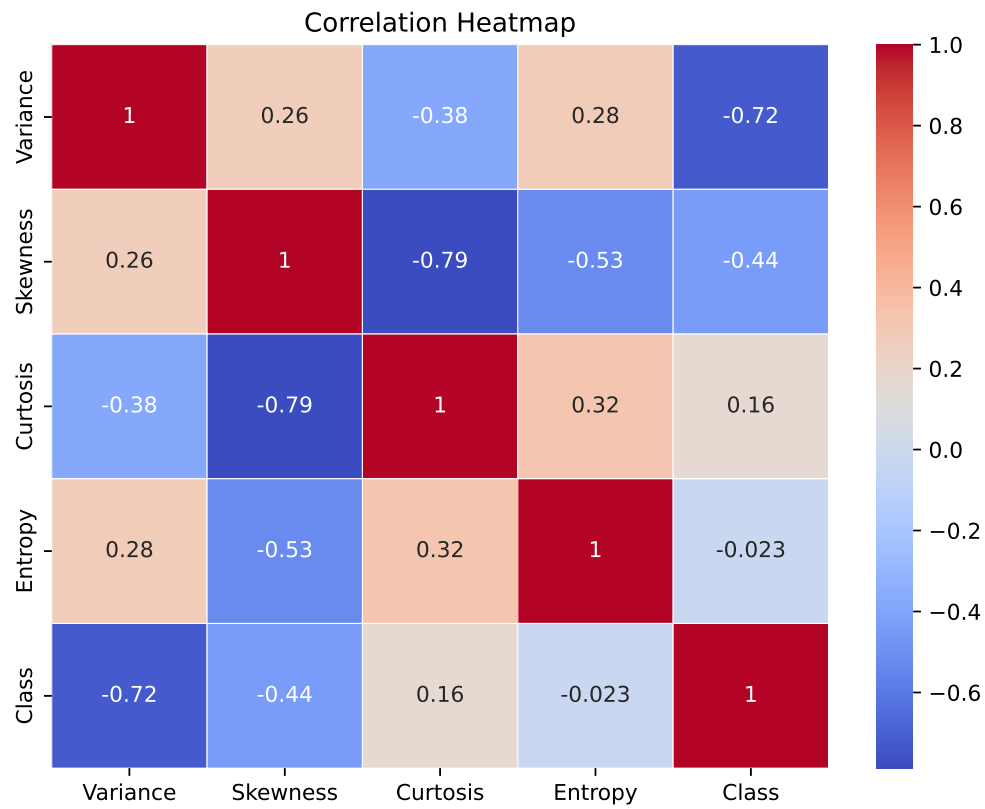


Figure 2: Correlation Heatmap

**Inference:** For positive values (like 0.32), when one variable goes up, the other variable also increases. For negative values (like -0.72), when one variable goes up, the other goes down. For numbers close to 0, like -0.023, the variables imply almost no relationship between the 2 variables.

### 3. Scatter Plots

#### Variance vs Skewness

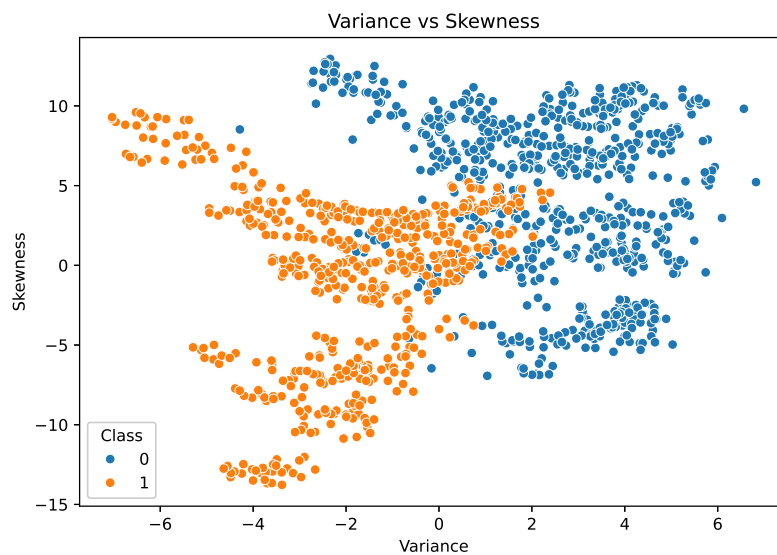


Figure 3: Variance vs Skewness Scatter Plot

**Inference:** If we draw a line from top left to bottom right, we can get most of the data points of blue above and orange below. Its not perfectly separable. We can infer that this is approximately linearly separable.

### 4. Training Error vs Epoch

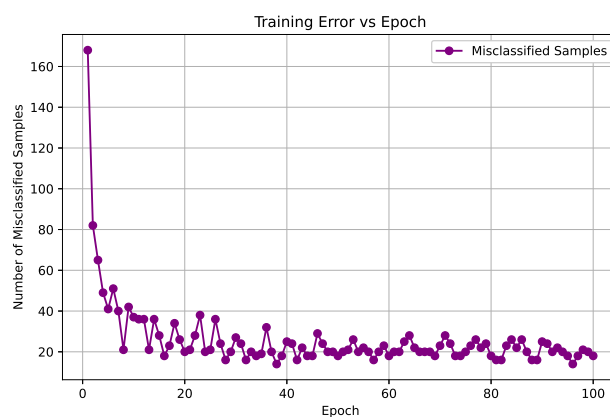


Figure 4: Training Error vs Epoch

**Inference:** The number of misclassified samples reduces/ seems to reduce after 20 epochs. It converges here.

## 5. Weight Evolution

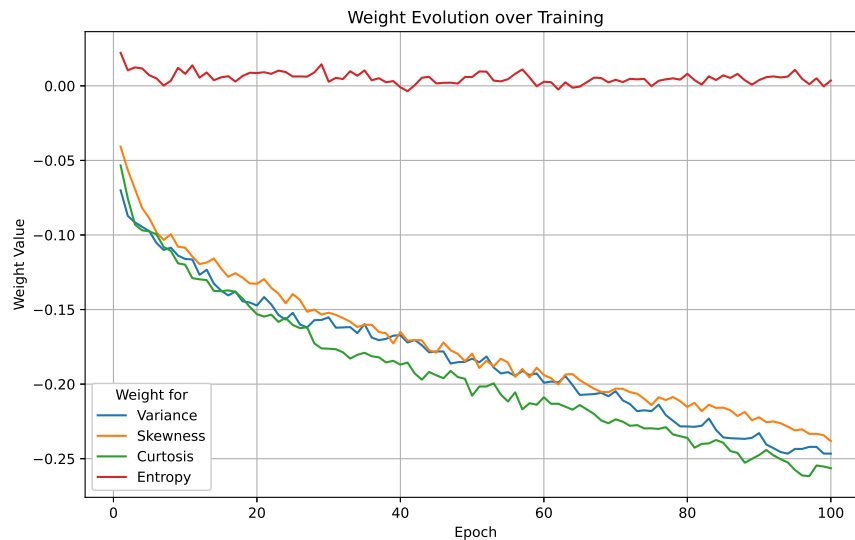


Figure 5: Weight Evolution

**Inference:** The weights for Variance, Skewness, and Kurtosis all show significant initial adjustments, so the model can find its optimal decision boundary. The weight for Entropy stays around 0 the entire time (it doesn't influence/ has very insignificant influence on model's decision).

## 6. Bias Evolution

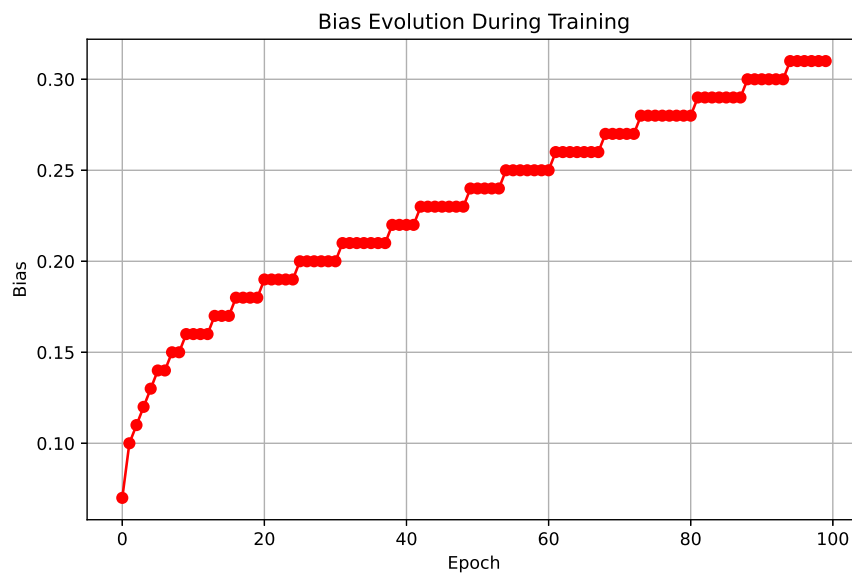


Figure 6: Bias Evolution

**Inference:** The bias value steadily goes up step-by-step from 0.0 to 0.3100 over the 100 epochs. This gradual increase helps shift the threshold so the perceptron can separate the two classes properly.

## 7. Confusion Matrix

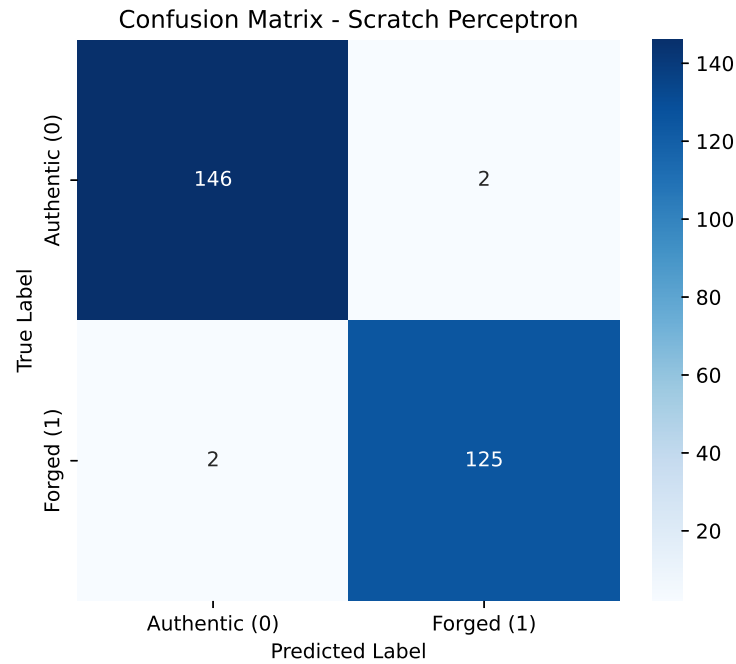


Figure 7: Confusion Matrix

**Inference:** Out of 275 test samples, the model shows a large number of True Positives and True Negatives (146 TN, 125 TP, 2 FP, 2 FN). 271 out of 275 samples were predicted correctly, giving a pretty good accuracy, with minimal error.

## 8. Learning Rate Comparison

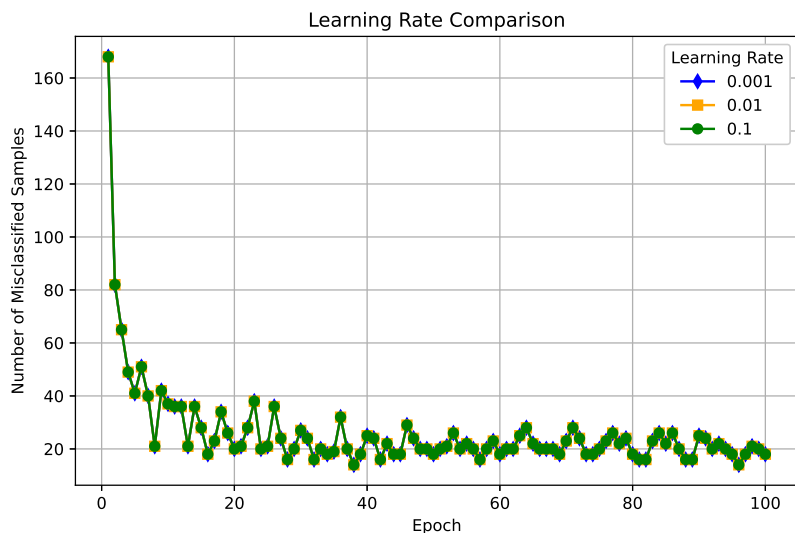


Figure 8: Learning Rate Comparison (Error vs Epoch)

**Inference:** All 3 learning rate lines (0.001, 0.01, and 0.1) completely overlap on the plot. Since the weights are initialised to zero and we are using a step function, changing the learning rate just multiplies the weight values by a constant factor. The sign of the output stays the same, so the actual decision boundary and total error count don't change at all.

## 9. Decision Boundary (Variance vs Skewness)

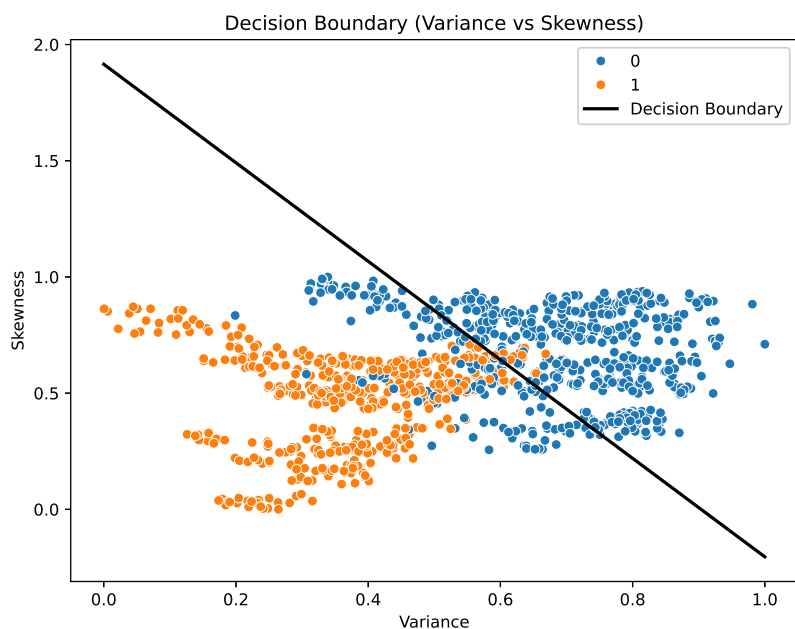


Figure 9: Decision Boundary on Variance vs Skewness

**Inference:** The plot shows the decision line splitting the data points fairly cleanly. Most authentic banknotes end up on one side while forged ones fall on the other. There are some overlapping points around the middle area, but overall the boundary isolates the 2 classes pretty well.

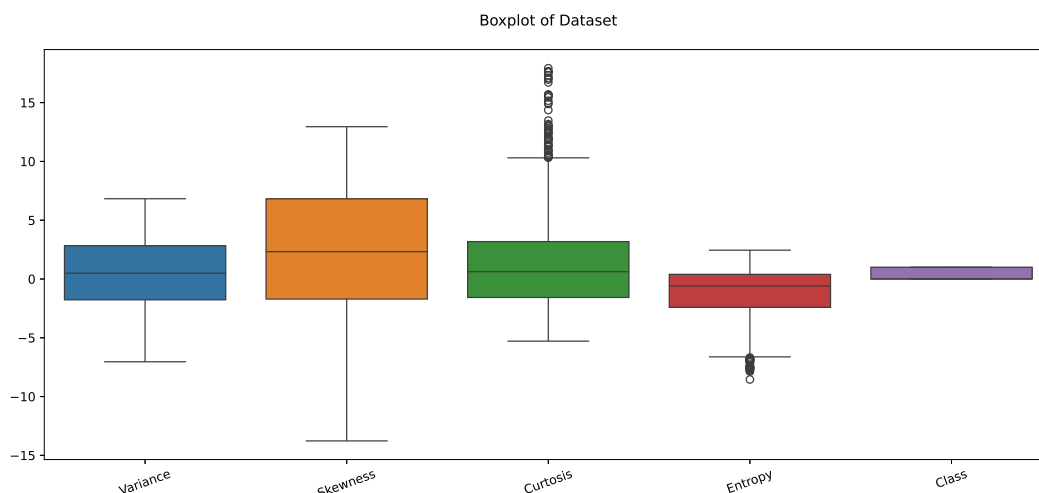


Figure 10: Boxplot Distribution

**Inference:** Variance is fairly balanced/symmetric around 0. Skewness shows the widest range of values across the dataset. Kurtosis has a lot of upper outliers. Entropy has a few lower outliers. Since the features operate on different scales, feature normalization is needed before training.

## Performance Tables

### Training Summary

|                  |                                        |
|------------------|----------------------------------------|
| Dataset Size     | 1372 samples                           |
| Train/Test Split | 1097 Training (80%), 275 Testing (20%) |
| Learning Rate    | 0.01                                   |
| Epochs           | 100                                    |
| Final Weights    | [-0.2476, -0.2284, -0.2534, 0.0074]    |
| Final Bias       | 0.3100                                 |
| Accuracy         | 98.55%                                 |
| Precision        | 98.43%                                 |
| Recall           | 98.43%                                 |
| F1-score         | 98.43%                                 |

### Epoch-wise Learning

| Epoch | Errors | Weight 1 | Weight 2 | Bias   |
|-------|--------|----------|----------|--------|
| 1     | 168    | -0.0697  | -0.0409  | 0.0800 |
| 50    | 21     | -0.1852  | -0.1813  | 0.2400 |
| 100   | 18     | -0.2464  | -0.2372  | 0.3200 |

## Additional Task: Logic Gates

1. Code the Perceptron Learning Algorithm for AND, OR and NOT gates using Python. Show the weights after each update and plot the decision boundary after each weight update.

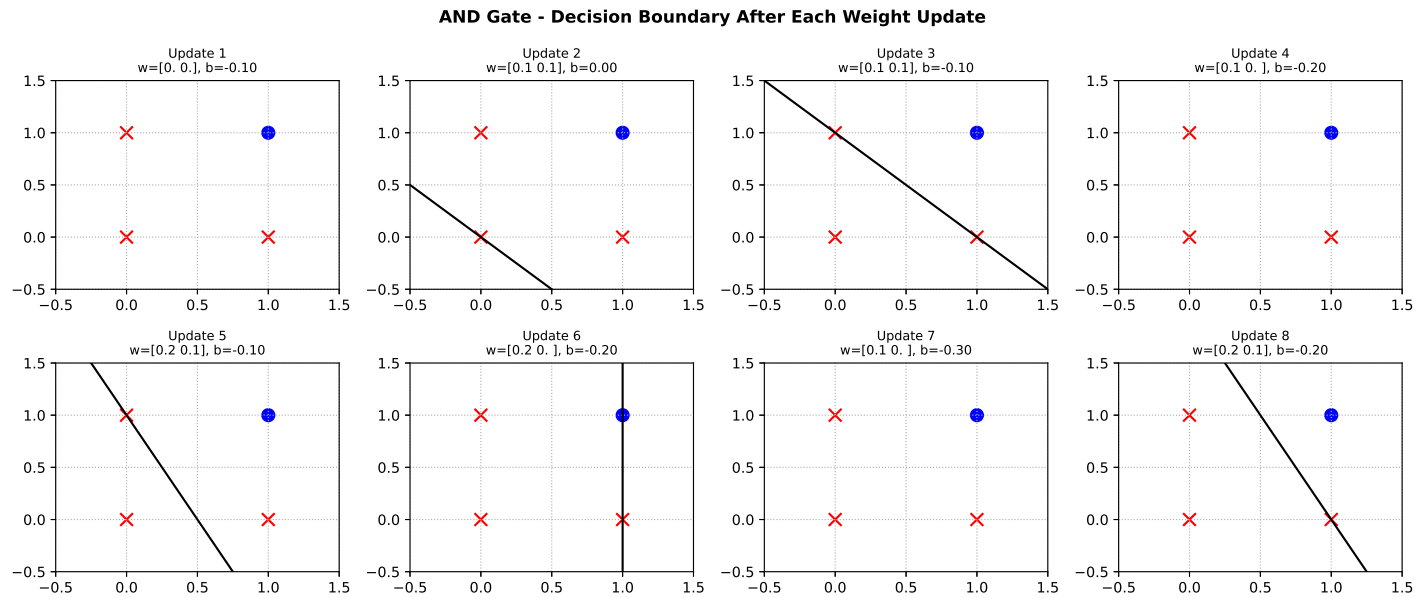


Figure 11: Decision Boundary Updates for AND gate

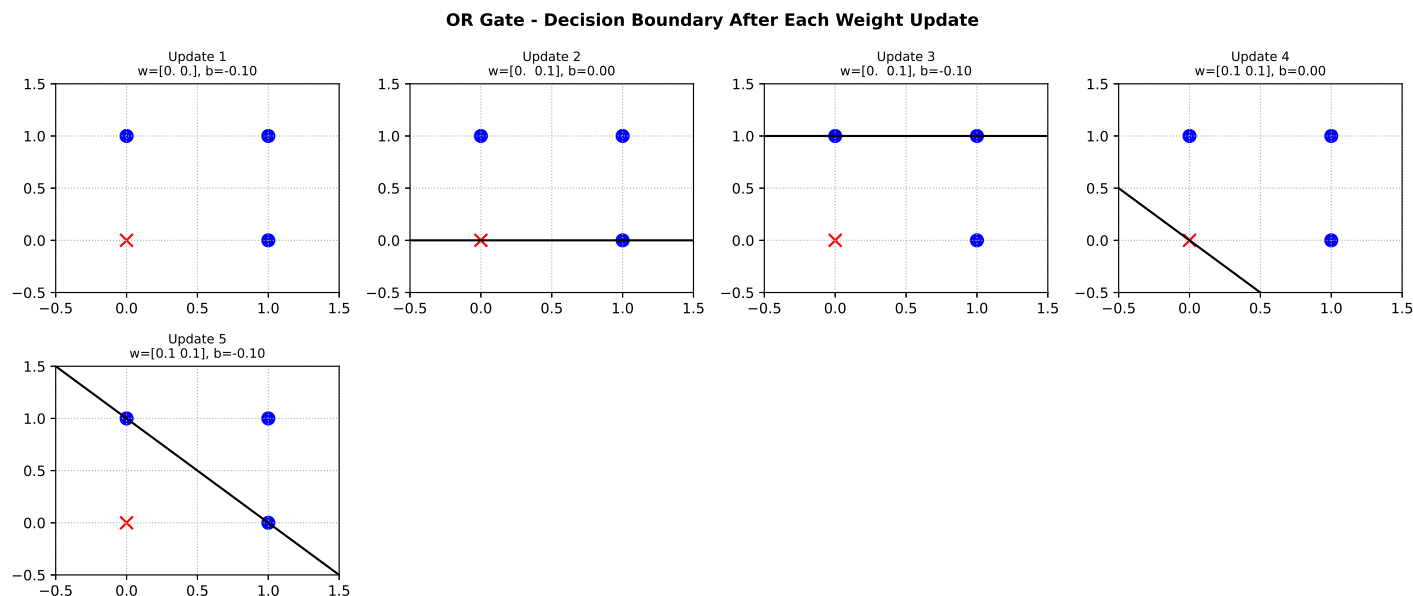


Figure 12: Decision Boundary Updates for OR gate

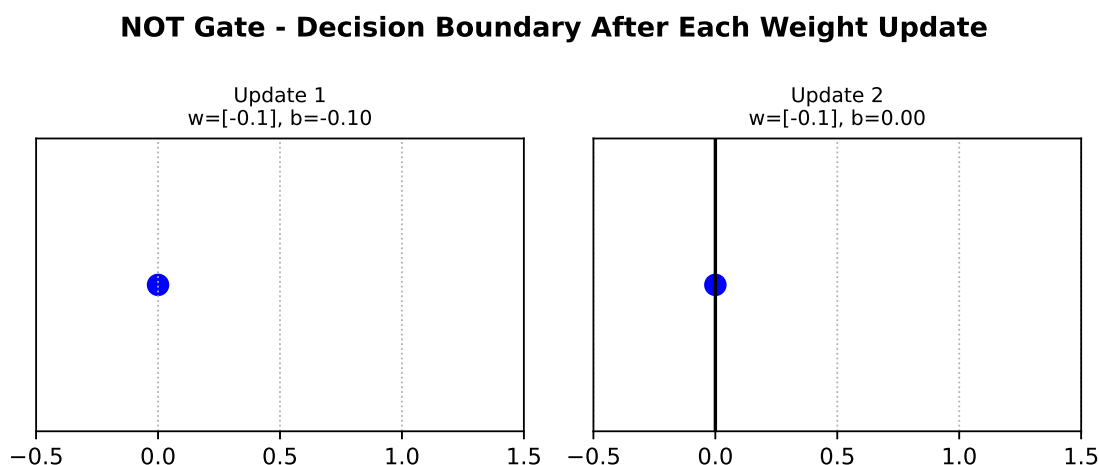


Figure 13: Decision Boundary Updates for NOT gate

## Additional Tasks

**Task 1: Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.**

**Comparison:**

- **Step Function:** Returns binary outputs(0 or 1). Derivative is 0 everywhere except  $x=0$  (it is undefined at  $x=0$ ).
- **Sigmoid Function:** Maps inputs into probabilities between 0 and 1. Calculated as  $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ .

### **Why Step Function is not used in Deep Learning:**

- Deep learning uses backpropagation and gradient descent to train weights.
- Gradient descent requires a proper, non zero derivative which is not equal to zero to propagate the error backward.
- The derivative of the step function is equal to zero everywhere, so the weights cannot be trained anymore.
- So we use smooth functions like Sigmoid, Tanh, or ReLU.

### **Task 2: Compare the implementation with Scikit-learn's Perceptron.**

- **Logic:** Both use Rosenblatt's weight update rule.
- **Performance:** When given initial zero weights, feature scaling and identical random seeds, both reach identical final accuracy.
- **Differences:** sklearn automatically learns early stopping, regularisation parameters, learning rate decay schedule, etc..

### **Task 3: Repeat the experiment using learning rates 0.001, 0.01 and 0.1.**

- **Result:** The error path does not change across the 3 learning rates (0.001, 0.01, and 0.1). Accuracy also doesn't change.
- Reason 1: Initial weights and bias were set to 0.
- Reason 2: The step activation only checks the sign of  $w \cdot x + b$ .
- **Reason 3:** Changing learning rates only rescales all the weights equally, thus leaving the decision boundary unchanged.

### **Task 4: Explain why a Single Layer Perceptron cannot solve the XOR problem.**

- A single-layer perceptron can only draw straight lines- linear decision boundaries.
- Inputs (0,0) and (1,1) give 0. Inputs (1,0) and (0,1) give 1. The 1s and 0s are in 2D space from each other. therefore, to solve XOR problem, we need a non-linear decision boundary (We require MLP with atleast 1 hidden layer.)

### **Task 5: Study the effect of feature normalization on model convergence.**

- **Without Normalization:** Features with large ranges dominate over features over small ranges. Results in unstable training.
- **With Normalization:** Every feature gets equal weight when scaled into a uniform range like [0,1].
- **Convergence:** Normalization keeps optimization stable, smoothens weight updates, and helps the model converge accurately.

**Conclusion:** This activity evaluates the performance and behaviour of a perceptron model. By implementing a Single Layer Perceptron from scratch, this activity provided a hands-on look at how artificial neurons function in binary classification.

## References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.